Architectural Optimization of 'La Ghigliottina' Solver

Stefano Colella
Master Degree in Computer Science - AI, University of Bari Aldo Moro

Abstract

This paper describes the development of an Information Retrieval and Reranking system designed to solve the Natural Language Processing task known as "La Ghigliottina". The research explores the potential of Distributional Machine Learning, converging towards a hybrid LTR (Learning-to-Rank) architecture. The structural integration of a Lexical Knowledge Graph (LKG) with an XGBoost Reranker successfully increased the Accuracy @1 from 21.00% to 33.00%, overcoming the physiological limitations of the individual paradigms and demonstrating strong resilience to overfitting. Furthermore, an Explainable AI (XAI) module based on Large Language Models was implemented to generate zero-shot logical reasoning, which was subsequently validated using NLG metrics.

Keywords

La Ghigliottina, Information Retrieval, Learning-to-Rank, XGBoost, Explainable AI, Lexical Knowledge Graph

Introduction and Motivations

This project aims to develop an Artificial Intelligence agent capable of solving the popular Italian television language game "La Ghigliottina". The task is structured as a lexical association problem: the system receives 5 seemingly unrelated clue words (hints) and must infer a single target word that shares a semantic, syntactic, or cultural connection with each of them.

To achieve State-of-the-Art (SOTA) performance, the research explored two opposing Artificial Intelligence paradigms: deterministic logical extraction from structured dictionaries (Symbolic AI) and statistical extraction from massive unstructured corpora (Distributional Machine Learning). The scientific motivation behind this approach lies in the need to overcome two well-documented bottlenecks in the literature: the Knowledge Bottleneck of purely deterministic systems (which fail to generalize nuanced connections) and the Entropic Noise inherent in the massive ingestion of encyclopedias and free text.

Related Work

"La Ghigliottina" task is widely documented in Italian academic literature, particularly through the EVALITA evaluation frameworks (e.g., the NLP4FUN task). State-of-the-art analysis highlights that systems relying purely on knowledge bases (Ontologies, Dictionaries) guarantee high precision on multi-word expressions but suffer from low recall, whereas purely distributional systems (Word Embeddings, PMI) suffer from semantic signal dilution. Literature theorizes that the combined Theoretical Upper Bound of these systems exceeds 70% accuracy. This work fits into the Retrieve-and-Rerank paradigm, proposing an orthogonal structural fusion between the two methodologies.

Proposed Approach

The infrastructure merges the two approaches into a unified Retrieve-and-Rerank pipeline to achieve the performance of a SOTA Hybrid Ensemble System, ensuring execution times compatible with real-time scenarios.

Description of the solution and dataset

The architecture relies on a dual data pool and a strict segmentation of the training dataset:

- Symbolic Knowledge: Local, highly qualified domain knowledge bases (demauro.poli, polirematiche, and proverbi), totaling 36,047 rows of exact lexical intersections.
- **Distributional Knowledge:** Over 11 Gigabytes of raw unstructured text extracted in an asymmetrical configuration (approximately 2.1M sentences from Corpus Paisà, 1.58M lines from Wikipedia, and 37.7M utterances from OpenSubtitles).
- Training and Evaluation Dataset: A set of historical games split into train.json and test.json (100 instances for the blind test set).

Main technical details

The system operates in sequential phases:

1. Deterministic Retrieval (LKG as a Filter): A Lexical Knowledge Graph based on an Inverted Index in volatile memory was engineered. The algorithm performs a Path-Based Reasoning search starting from the 5 hint-nodes with an $\mathcal{O}(1)$ access time. The LKG acts as an ultra-high recall primary filter, extracting a restricted pool of candidates (Top 50).

2. 'SparseCooccurrenceEngine' and Matrix Pruning: Distributional texts are processed via direct indexing. To prevent Out-Of-Memory (OOM) errors caused by the theoretical $\mathcal{O}(V^2)$ dense matrix footprint (which would exceed 10 Terabytes of RAM for a 1.6M vocabulary), tensors are offloaded in a compressed format and algebraically summed into a master 'scipy.sparse.csr_matrix'.

3. Supervised Reranking (XGBoost): The vector space for each extracted candidate is defined in $\mathbb{R}^8$. The Learning-to-Rank model dynamically balances the strong logical features extracted from the Symbolic Graph with the historical distributional metric. XGBoost was parameterized to penalize the NPMI when in conflict with an exact topological intersection, algebraically encoding the lack of co-occurrence as -1.0 (semantic repulsion).

Other information useful to replicate the approach

A critical architectural element for reproducibility concerns the prevention of Data Leakage. The 'train.json' dataset was subjected to a strict Hold-out Split (80/20). The 80% partition populates the Graph with explicit television history labels, providing long-term memory. The remaining 20% undergoes Retrieval for the binarized training of the XGBoost trees. Without this orthogonal separation, the model would artificially saturate the Information Gain on the historical feature, blinding itself to pure distributional statistics.

Finally, a cloud-based Explainable AI (XAI) module was integrated using the asynchronous Groq APIs (model LLaMA-3.3-70B). To prevent Rate Limiting (Error 429), the module implements an Asynchronous Throttling logic with a calibrated 1.5-second delay per request, ensuring a continuous and stable throughput.

Operational Logic and Data Structure Optimization

Processing over 11 GB of raw unstructured text and scaling a 'Retrieve-and-Rerank' architecture necessitates rigorous software engineering optimizations. The system was engineered to mitigate critical bottlenecks related to volatile memory (RAM) allocation, algorithmic time complexity, and API rate limiting.

Streamed Data Ingestion and Text Normalization

To prevent system crashes during the parsing of massive corpora (such as the 37.7 million utterances from OpenSubtitles), the ingestion pipeline implements lazy evaluation via Python generators. Text is processed line-by-line rather than being loaded into memory entirely.

Text normalization relies on dynamic set operations: the baseline vocabulary is fetched from the NLTK corpus, extended with task-specific particles, and strictly filtered using a set difference operation (BASE. union(CUSTOM) - VITALI). This set-theoretic approach ensures $\mathcal{O}(1)$ lookup times during the stopwords removal phase, significantly accelerating the tokenization throughput.

High-Dimensional Matrix Compression (CSR)

The most severe architectural challenge involved computing the Pointwise Mutual Information (NPMI) across the distributional dataset. With an extracted vocabulary V of approximately 1.6 million lemmas, allocating a standard dense matrix in 32-bit floating-point precision to track co-occurrences would require:

$$\mathbf{1.6 \times 10^{6} \times 1.6 \times 10^{6} \times 4 \approx 10.24 \ \text{Terabytes of RAM}}$$

To circumvent this Out-Of-Memory (OOM) certainty, the system utilizes Compressed Sparse Row (CSR) matrices via scipy.sparse.csr_matrix. This data structure dynamically allocates memory solely for non-zero elements. Combined with mathematical pruning where co-occurrences with a frequency $f_{xy} < 2$ are dropped as entropic noise, the topological footprint is compressed from over 10 Terabytes of theoretical dense space to just a few Gigabytes of active edges, making the system viable for consumer hardware.

Algorithmic Complexity and $\mathcal{O}(1)$ Lookups

Both the Symbolic Lexical Knowledge Graph (LKG) and the Distributional Co-occurrence Engine are structured around Inverted Indexes and Hash Maps (word2id dictionaries).

During the inference phase, the retrieval algorithm must branch out from the 5 given hints. The use of pre-computed hash maps guarantees that fetching a node's topological degree, its adjacent edges, and its marginal probability $p(x)$ occurs in strictly constant time $\mathcal{O}(1)$, eliminating nested iterations and ensuring an average game resolution time of less than 0.15 seconds.

Asynchronous Pipeline and API Throttling

The Explainable AI (XAI) generation relies on Cloud LLM inference (LLaMA-3.3-70B). Processing batches of games synchronously would result in severe network latency and trigger HTTP 429 (Too Many Requests) errors from the provider.

The operational logic was therefore decoupled using the asyncio library. The system spawns asynchronous coroutines managed by an event loop, implementing a strict throttling mechanism (a calibrated 1.5-second minimum delay per task). This ensures a continuous, stable throughput without saturating the API rate limits, seamlessly integrating the heavy generative module into the fast retrieval pipeline.

Evaluation

The models were validated on a blind Test Set, evaluating Exact Match (Accuracy @1), candidate window recall (Accuracy @5), and Mean Reciprocal Rank (MRR).

Symbolic and Distributional Baselines

Isolating the Symbolic approach (LKG only) yielded an Accuracy @1 of 21.00% and an Accuracy @5 of 42.00% (MRR: 0.2957). This demonstrates the retrieval effectiveness on strong associations (e.g., idioms) but confirms the vulnerability to the Knowledge Bottleneck for latent semantic associations. Conversely, the pure Distributional approach (NPMI only) confirmed the theoretical entropy limit: the asymmetrical corpora configuration capped Accuracy @1 at 19.00-21.00%, with Accuracy @5 collapsing to 27.00%.

SOTA Hybrid Model and XAI Module

Integration into the Ensemble Hybrid model (LKG + XGBoost) produced a sharp outperformance over the individual baselines:

- Accuracy @1 (Exact Match): 33.00% (+12.00% absolute vs. symbolic baseline).
- Accuracy @5 (Top-5 Window): 49.00% (+7.00% absolute vs. symbolic baseline).
- Mean Reciprocal Rank (MRR): 0.3987.
- Average Inference Time: < 0.15s per game.

Analysis of the Information Gain extracted from the decision trees provides mathematical proof of proper regularization: the web-based statistical feature (f_8) absorbed 16.21% of the decision weight, harmoniously joining the topological features (f_1: 30.48%, f_3: 24.32%).

The XAI module successfully generated the explanatory Ground Truth in a zero-shot setting. Post-hoc NLG evaluation against human-written explanations returned:

- BLEU Score (Precision): 0.1069
- ROUGE-L F1 (Recall): 0.3265
- BERTScore F1 (Vector Semantics): 0.7636

The BERTScore value attests to an excellent semantic alignment (0.76) between the LLM-generated explanations and human associative logic. The observed divergence between the high BERTScore and the lower BLEU and ROUGE-L metrics is an expected outcome of the prompt engineering strategy and highlights the structural limitations of n-gram-based evaluation. The Large Language Model was instructed via few-shot prompting to generate comprehensive, discursive explanations. Consequently, the generated text introduces explanatory tokens absent from the highly concise, telegraphic human-written ground truth. This verbosity heavily penalizes exact string-matching metrics like BLEU and ROUGE. Conversely, the Multilingual BERTScore bypasses string-matching constraints by computing pairwise cosine similarity on contextual token embeddings, successfully capturing the associative logic.

Conclusion and limitations

The performance leap from 21.00% to 33.00% in Exact Match unequivocally demonstrates the research's structural thesis: pure syntactic knowledge requires distributional regularization to disambiguate the complex polysemy demanded by lateral thinking. The integration of Data Splitting, Feature Injection, and Asynchronous Throttling techniques resulted in a robust real-time solver. The primary limitation lies in the computational dependency required to calculate and host massive NPMI matrices in RAM—an acceptable engineering trade-off given the achieved results.

References

[1] Basile, P., et al. "NLP4FUN: Task description and results." Proceedings of the EVALITA 2018 Evaluation of NLP and Speech Tools for Italian (2018).

[2] Chen, T., & Guestrin, C. "XGBoost: A scalable tree boosting system." Proceedings of the 22nd ACM SIGKDD (2016).

[3] Zhang, T., et al. "BERTScore: Evaluating Text Generation with BERT." ICLR (2020).